News sentiment and daily equity volatility forecasting

A mathematical and implementation handout

Dhruv Natu. Project documentation and reproducible audit, 21 October 2026.

This project asks whether financial-news sentiment improves the next trading day's conditional variance forecast beyond the information already captured by a GARCH model. It combines daily equity prices, pretrained financial-language-model inference, a custom conditional-variance likelihood, chronological forecasting, and forecast-loss comparisons. It is an empirical forecasting study, with no trading strategy or causal identification design.

The evidence is mixed. The original cached experiment gives lower average QLIKE to GARCH-X for three of ten stocks. After corrections to timing, constraints, initialization, preprocessing, and evaluation, GARCH-X gives lower average QLIKE for four of nine eligible stocks in the main experiment. NVIDIA provides an exploratory improvement relative to GARCH under both the main and alternative chronological splits, but its EWMA benchmark is better on main-split QLIKE. This is evidence against claiming a general forecasting advantage from the particular sentiment feature tested here.

The corrected experiment remains conditional on imperfect cached news coverage and historical publication timestamps. Software repairs do not reconstruct missing articles, prove that archived text was available at its recorded creation time, or turn a retrospective study into an untouched prospective test.

Contents

How to use this handout 3

1 Research question and the contribution 3

2 The forecast target and the information set 3

3 Prices returns and units 4

4 Trailing volatility is a descriptive statistic 5

5 News collection and the coverage audit 5

6 Turning headlines into a sentiment regressor 7

7 Why GARCH is the baseline 8

8 The signed sentiment GARCH X extension 9

9 Gaussian likelihood and quasi maximum likelihood 10

10 Initialization optimization and numerical checks 11

11 Parameter uncertainty and the original robust covariance code 12

12 Chronological forecasting without future returns 13

13 Evaluation targets loss functions and benchmarks 13

14 Diebold Mariano comparisons and multiple testing 14

15 Empirical findings 16

16 Parameters diagnostics and interpretation 18

17 Implementation map 20

18 Corrections made and their scope 21

19 Reproduction and verification 22

20 Interview answers and self assessment 22

Appendix A A complete numerical example 24

Appendix B Notation and units 25

Appendix C Questions for further work 25

References 26

How to use this handout

Read Sections 1 through 6 first to understand the target, information set, data, and sentiment feature. Sections 7 through 12 develop the model, constraints, likelihood, estimation, forecasting, and statistical inference. Sections 13 through 16 explain evaluation, empirical results, diagnostics, and limitations. Sections 17 through 20 map the theory to files, document corrections, provide reproduction commands, and give interview answers. The appendices collect notation, numerical examples, and further study topics.

The terms original and corrected have precise meanings here. Original refers to the twelve Python scripts archived before this audit and their existing root-level CSVs. Corrected refers to the new shared modules and the generated files under output. The existing source and data were preserved before changes; the handout never presents newly introduced restrictions or calendar alignment as choices already present in the original experiment.

1 Research question and the contribution

The hypothesis is incremental predictive information. Yesterday's returns already contain information about recent shocks and volatility. News may contain additional information about uncertainty, earnings, litigation, product developments, or financial conditions. The experiment asks whether a numerical summary of that news improves forecasts after conditioning on a baseline GARCH recursion.

The ten original tickers are AAPL, MSFT, NVDA, AMZN, GOOGL, META, BRK-B, LLY, AVGO, and TSLA. These are a selected set of large US equities, rather than a random sample of the equity market. The list has no documented historical selection rule. Consequently, conclusions apply to these assets, this period, this feature construction, and these forecasting procedures; they do not automatically generalize to all equities.

The work consists of assembling the pipeline, applying an existing financial NLP model, implementing and estimating a GARCH-X extension, and comparing held-out forecast losses. It does not introduce the GARCH family, the FinBERT architecture, or the Diebold-Mariano test. It also does not train or fine-tune FinBERT. Being explicit about these boundaries makes the project easier to defend.

The central distinction is between association, prediction, and causation. A nonzero fitted sentiment coefficient describes a conditional association under the chosen specification. A smaller held-out loss measures predictive performance under a particular loss. Neither establishes that changing sentiment would cause volatility to change. News and volatility can respond to the same underlying event.

2 The forecast target and the information set

Let t index exchange trading sessions rather than arbitrary calendar dates. Let P_t be the cached closing-price series, and let R_t denote the percentage log return from the preceding session's close to this session's close. The conditional mean and variance are defined relative to the information available at the forecast origin.

$$R_t = 100 \log\left(\frac{P_t}{P_{t-1}}\right)$$

$$R_t = \mu + \varepsilon_t, \qquad \varepsilon_t = \sqrt{h_t}\, z_t$$

$$E[z_t \mid \mathcal{F}_{t-1}] = 0, \qquad E[z_t^2 \mid \mathcal{F}_{t-1}] = 1$$

$$h_t = \mathrm{Var}(R_t \mid \mathcal{F}_{t-1})$$

The model forecasts h_t, a conditional variance in squared percentage-return units. Daily volatility is its square root. The direction of the return is a separate target. A high variance forecast permits either a large gain or a large loss.

The corrected operational convention is an issuance time C_t equal to the actual exchange close plus five minutes. On a normal session that is 16:05 America/New_York; on a 13:00 early-close session it is 13:05. Calendar conversion respects daylight saving changes. The five-minute buffer is a transparent research convention intended to permit acquisition of the closing observation; actual vendor latency has not been measured. The XNYS calendar is used for these US equities as a common session-close calendar. [S6, S7]

At C_{t-1}, the next-session forecast may use the completed return R_{t-1}, prior filtered variances, and news recorded no later than C_{t-1}. It may not use R_t, subsequent articles, or revised historical text whose earlier version is unavailable. Some first minutes of the close-to-close holding interval have already elapsed by issuance. Therefore the experiment is most precisely a forecast of the next labeled close-to-close daily return at a post-close origin; it is not an exact 24-hour-ahead forecast from the issuance timestamp.

For a forecast issued Friday after the close, Saturday and Sunday articles are not yet known. Incorporating them into Monday's forecast would require a later issuance convention and a consistent redefinition of the information set. In this implementation, weekend stories become available at Monday's issuance and can affect Tuesday's forecast. That is a consequence of the chosen clock, not an accidental omission.

3 Prices returns and units

The price cache spans 5 June 2020 through 4 June 2026. The original download request uses 5 June 2026 as its end argument; the actual maximum observation in the supplied file is the preceding session. Always report observed coverage rather than infer it from the request parameters.

The original code computes log(1 + pct_change()). For strictly positive prices without implicit filling this is algebraically the same as the difference of log prices. The corrected implementation uses np.log(close).diff(), making the treatment of missing prices explicit and avoiding forward-filled returns.

Log returns add over time: the sum of consecutive log price ratios equals the log ratio between the final and initial prices. A simple return instead compounds multiplicatively. For small movements log and simple returns are numerically close; for large movements the distinction matters.

The multiplication by 100 is numerical rescaling, not standardization. A decimal log return of 0.02 becomes approximately 2 percentage points. The variance is multiplied by 100 squared. Alpha and beta are dimensionless, while mu has percentage-return units and omega and gamma have squared percentage-return units because sentiment is dimensionless.

$$R_t = 100 r_t, \qquad h_t^{(R)} = 10^4 h_t^{(r)}$$

$$\sigma_{t,\,\text{annual}} \approx \frac{\sqrt{252}\sqrt{h_t^{(R)}}}{100}$$

The annualization formula reports a decimal annualized volatility under a trading-day scaling convention. It is a display convention for a one-day variance forecast, not a fitted annual model. Serial dependence, time-varying variance, and a future path of shocks complicate the variance of a full year of returns.

A daily variance forecast of 4 implies daily volatility of 2 percentage points and approximately 31.75 percent annualized volatility. It does not imply an expected return of 2 percent. Multiplying a return by 100 also changes a Gaussian likelihood by a units-dependent constant; likelihood comparisons require the same units and observations.

The original yfinance call does not explicitly specify auto_adjust or store the download library version. Current library defaults cannot prove how this existing cache was created. Its adjustment provenance is therefore unresolved. The corrected analysis keeps the cache immutable rather than silently redownloading prices with different adjustment choices. A future collection should record whether prices are split-adjusted, dividend-adjusted, or raw, plus the vendor, retrieval date, arguments, and version. [S8]

4 Trailing volatility is a descriptive statistic

The original price module also computes a trailing 21-session standard deviation of decimal log returns, annualized by the square root of 252. With m equal to 21 and the sample-variance denominator m minus one, its definition is:

$$\bar{r}_{t,m} = \frac{1}{m} \sum_{j=0}^{m-1} r_{t-j}$$

$$v_{t,21} = \sqrt{252} \sqrt{\frac{1}{20} \sum_{j=0}^{20} (r_{t-j} - \bar{r}_{t,21})^2}$$

This is backward-looking historical volatility. It is neither a one-day conditional variance nor an intraday realized-variance estimator. In financial econometrics, realized variance often means a sum of squared high-frequency returns within a day; that estimator is not computed in this project.

The original merge drops early observations because this trailing statistic is missing, even though the GARCH likelihood only needs valid returns and sentiment. The corrected experiment retains those usable returns and leaves trailing volatility as a descriptive column. This changes the main sample from 1,486 original merged returns to 1,505 corrected rows before ticker-specific news-envelope screening.

The forecast evaluation target is a squared daily return residual. It should not be confused with v_{t,21}, and directly subtracting annualized decimal volatility from daily percentage variance would be dimensionally invalid.

5 News collection and the coverage audit

The original downloader requests Alpaca historical news by ticker, in batches of 50, then repeatedly moves the end time to the last returned article's created_at timestamp. The API documents inclusive time boundaries, token pagination, and ordering by updated time. Combining an inclusive creation-time boundary with that ordering can repeat observations and cannot guarantee complete retrieval. The stored date/headline pairs contain duplicates. [S4]

The corrected downloader delegates token pagination to the installed Alpaca SDK with limit=None. Its installed implementation follows next_page_token while requesting valid page sizes. It preserves article ID, created_at, updated_at, headline, and symbols, deduplicates by article ID, and writes a new snapshot rather than replacing the supplied historical cache. The corrected collection path was inspected and import-checked; no credentialed refresh was executed during this audit. [S5]

The old cache lacks article IDs and update histories. Its deterministic repair removes duplicate date/headline pairs, keeping the first stored sentiment score. This is a practical approximation: distinct articles with identical timestamp and headline could be merged, while syndicated stories with different wording or timestamps remain separate. It cannot recover omissions introduced during collection.

Ticker	Articles	Duplicates	First article	Last article	Zero lag %
AAPL	21,343	169	2020-06-05	2026-06-04	2.2
MSFT	15,749	116	2019-09-17	2026-06-04	0.5
NVDA	17,518	107	2017-07-31	2026-06-04	4.6
AMZN	17,735	126	2018-03-26	2026-06-04	0.0
GOOGL	14,476	105	2018-03-26	2026-06-04	0.3
META	9,477	54	2021-06-30	2026-06-04	38.2
BRK-B	442	4	2014-10-20	2023-10-06	81.9
LLY	3,741	20	2020-06-05	2026-06-04	20.1
AVGO	3,385	20	2020-06-05	2026-06-04	32.7
TSLA	17,036	131	2020-06-05	2026-06-04	38.0

Table 1. Original sentiment-cache coverage and duplicate audit. Zero lag percentages describe the old merged data, not verified neutral-news frequency. News counts include records outside the requested price period where present.

BRK-B is excluded from corrected full-period comparisons because its news cache stops on 6 October 2023 while the price series continues to June 2026. META starts much later than the price cache, so corrected modeling starts only when a full prior news window lies inside its observed news envelope. Different asset-specific start dates mean the comparisons are not a balanced panel.

The timestamp envelope is only an edge screen. It establishes that a window lies after the first and before the last stored article. It does not establish that collection was complete inside that range. In particular, TSLA has a 236-day gap between observed news dates, AAPL has a 45-day gap, and META has a 63-day gap. For heavily covered firms these gaps raise substantial concerns about missing collection, but the cache alone cannot determine the cause.

Ticker	Gap days	From news date	To news date
AAPL	45	2022-11-14	2022-12-29
AMZN	4	2021-12-23	2021-12-27
AVGO	29	2021-03-15	2021-04-13
BRK-B	25	2020-07-16	2020-08-10
GOOGL	5	2020-07-02	2020-07-07
LLY	14	2020-12-22	2021-01-05
META	63	2021-08-26	2021-10-28
MSFT	4	2020-07-02	2020-07-06
NVDA	7	2020-10-15	2020-10-22
TSLA	236	2023-03-09	2023-10-31

Table 2. Longest observed gaps within the price period. The 2020 and 2026 year counts in the machine-readable audit cover partial years. A gap is elapsed time between dates with articles, not a confirmed outage measurement.

The corrected feature records news_count_lag1 separately. An empty window has a zero mean-score convention, but this means no articles in this cache, not proven market neutrality. The current variance equation does not include that count as another regressor; retaining it makes missingness and future feature extensions inspectable.

An additional coverage sensitivity keeps only the contiguous final period after each ticker's last gap exceeding 30 calendar days. This avoids concatenating returns across gaps. Thirty days is a heuristic audit threshold, not a statistical guarantee of complete coverage, and shorter missing intervals remain possible. This sensitivity uses different samples and must not be interpreted as a controlled single-factor experiment.

6 Turning headlines into a sentiment regressor

The project applies ProsusAI/finbert, a pretrained BERT-based financial sentiment classifier. A headline is tokenized into subword tokens, truncated if necessary, padded within its batch, and passed through the model. Batching 64 headlines improves throughput without changing the intended mathematical mapping. No task-specific parameter training is performed in this project. [S2]

Let a_i be the model logits for article i and let k range over positive, negative, and neutral. Softmax produces nonnegative scores summing to one:

$$p_{i,k} = \frac{\exp(a_{i,k})}{\sum_j \exp(a_{i,j})}$$

$$s_i = p_{i,+} - p_{i,-}, \qquad -1 \le s_i \le 1$$

These are model-assigned class probabilities, not independently validated forecasts of economic impact. Their calibration on this project's headlines has not been measured. A score near zero may reflect neutral text or an ambiguous mixture of positive and negative probability.

The original index mapping, column zero minus column one, agrees with this checkpoint's published positive and negative labels. That mapping was not itself a discovered bug. The corrected implementation reads labels from model.config.id2label rather than assuming the order. It explicitly sets evaluation mode, uses inference_mode, validates headline inputs, and loads the model lazily. Setting eval explicitly clarifies inference behavior; the audit does not assert that the original pretrained-loader default necessarily activated dropout. [S3]

For the issuance window from C_{t-2} exclusive to C_{t-1} inclusive, define the target-aligned feature x_t as the average article score. The name sentiment_lag1 in the data already refers to this feature for return t; the recursion must not lag it a second time.

$$I_t = \{i : C_{t-2} < \tau_i \le C_{t-1}\}, \qquad n_t = |I_t|$$

$$x_t = \frac{1}{n_t} \sum_{i \in I_t} s_i \quad \text{when } n_t > 0$$

For n_t equal to zero, x_t is set to zero and the count remains zero. An article exactly at issuance belongs to that issuance window. The corrected implementation compares timestamps in a common nanosecond representation; this is necessary because pandas can store parsed timestamps at microsecond resolution while exchange-calendar timestamps use nanoseconds.

Equal-weight averaging is simple, bounded, and interpretable, but it suppresses news intensity. One article and one hundred articles can have the same mean. Equally positive and negative stories can cancel despite substantial uncertainty. These are substantive feature-design limitations rather than arithmetic errors.

6 1 What the language model computes

BERT first represents tokens using learned embeddings and positional information. In a self-attention head, the current token representations H are projected into queries Q, keys K, and values V. Attention mixes value vectors using similarities between queries and keys, scaled by the square root of the key dimension. An attention mask prevents padding positions from acting as real words.

$$Q = HW_Q, \qquad K = HW_K, \qquad V = HW_V$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK'}{\sqrt{d_k}} + M\right)V$$

M denotes the additive mask, with padded positions receiving effectively negative infinity before softmax. Multiple attention heads permit different learned interactions, followed by feedforward transformations, residual connections, and layer normalization. A sequence-level representation is passed to a classifier that produces the three logits used above. This explanation describes the kind of pretrained model applied here; the project does not implement those layers from scratch.

The advantage over a dictionary word count is contextual interpretation. A financial phrase can have a different meaning depending on the surrounding words, negation, and the event described. That flexibility does not ensure correct interpretation of every headline, and a headline mentioning multiple firms may express different implications for each firm. The same headline-level score assigned to every mentioned ticker is a simplifying assumption of this dataset.

The token limit of 512 is a sequence budget, including the tokenizer's special-token conventions; it is not a 512-word limit. Padding makes batch tensors rectangular, truncation removes excess tokens, and moving tensors and parameters to the same device permits computation. Disabling autograd reduces memory use; evaluation mode controls training-dependent layers. Neither operation trains the model or calibrates its outputs on the project's data.

The pretrained network is fixed while the downstream GARCH-X parameters are estimated from the training period. This separation means that no supervised fine-tuning on held-out returns occurs, but it does not establish a fully historical NLP backtest: the precise cached checkpoint revision and article text versions were not stored originally. A future reproducible archive should pin the model commit as well as tokenizer and software versions.

Alternative features would test different hypotheses: absolute sentiment captures magnitude; separate positive and negative components permit asymmetry; article count measures attention; disagreement measures dispersion; uncertainty scores target uncertainty more directly. They should be specified and evaluated as additional models with validation and multiplicity control rather than selected after inspecting the test winners.

Historical created_at is only a proxy for availability. Without first-seen timestamps and archived text versions, a later revision may be scored as if its text existed earlier. Neither the corrected calendar nor FinBERT rescoring can remove this uncertainty from the existing cache. The audit reused cached scores; it did not rerun the optional GPU-dependent scoring stage.

7 Why GARCH is the baseline

Volatility clustering means large return magnitudes often occur near other large magnitudes even when signed returns have little linear autocorrelation. Constant-variance models ignore that structure. GARCH represents conditional variance as a function of a recent squared innovation and the previous conditional variance. The baseline is parsimonious and provides a strong comparison before adding text features. [S1]

$$h_t = \omega + \alpha\varepsilon_{t-1}^2 + \beta h_{t-1}$$

Omega is the variance intercept, alpha measures sensitivity to the previous unexpected squared return, and beta carries forward the previous variance state. Alpha does not multiply lagged variance in this project's parameter order. Epsilon is the return minus the estimated mean, not the return itself unless the mean is zero.

The GARCH(1,1) designation indicates one lag of squared innovations and one lag of variance. Conventions for the ordering of general p and q labels differ across presentations; the displayed equation removes ambiguity. The code's four parameters are ordered mu, omega, alpha, beta.

With omega positive, alpha and beta nonnegative, and positive initial variance, induction gives positive variance at every subsequent step. Under a correctly standardized innovation process and suitable stationarity assumptions, taking expectations gives:

$$E[h_t] = \omega + (\alpha + \beta)E[h_{t-1}]$$

$$\bar{h} = \frac{\omega}{1-\alpha-\beta} \quad \text{if} \quad \alpha + \beta < 1$$

This finite-second-moment condition is different from positivity. A model can be positive but have no finite unconditional variance. Strict stationarity also has a different condition, involving the logarithm of the random recurrence coefficient. In the standard iid-innovation setting, a sufficient contraction condition is:

$$E\left[\log(\alpha z_t^2 + \beta)\right] < 0$$

The stronger alpha-plus-beta condition implies this log condition under ordinary integrability assumptions through Jensen's inequality, but the reverse need not hold. A stationary process can have infinite moments. Finite fourth moments require stronger restrictions; for Gaussian iid standardized innovations in ordinary GARCH, a familiar condition is:

$$3\alpha^2 + 2\alpha\beta + \beta^2 < 1$$

Persistence is often summarized by alpha plus beta because future squared innovations also respond in expectation to the variance state. It is not generally correct to equate beta alone with the persistence of the expected shock response. For persistence p between zero and one, an expected-response half-life is log(1/2) divided by log(p), conditional on the usual model assumptions and no additional persistent exogenous response.

8 The signed sentiment GARCH X extension

The original extension adds the already target-aligned signed news feature linearly:

$$h_t = \omega + \alpha\varepsilon_{t-1}^2 + \beta h_{t-1} + \gamma x_t$$

The code parameter order is mu, omega, alpha, beta, gamma. Gamma measures the additive change in conditional variance per unit of signed sentiment, holding the previous shock and variance state fixed. A positive gamma makes positive sentiment raise variance and negative sentiment lower it. A negative gamma reverses that relation. This is a directional tone hypothesis; it does not automatically express the hypothesis that all intense news raises uncertainty.

The implementation does not square sentiment, take its absolute value, or normalize every term. Squaring it would change the scientific question and eliminate the sign distinction. The signed score remains bounded because each article score and its average lie between minus one and one.

The baseline is nested in the extension at gamma equal to zero. Therefore an adequately optimized training likelihood for the unrestricted extension cannot be worse than the same baseline likelihood evaluated with gamma zero, provided both share the same observations, initialization, mean treatment, and remaining restrictions. This fact concerns training fit, not held-out accuracy.

In the original code, positive variance is checked only along the realized fitted path. That does not guarantee a valid future forecast for an extreme but admissible sentiment value. The corrected model imposes a sufficient global condition. Since the minimum of gamma times x over the allowed interval is minus the absolute gamma:

$$\min_{x \in [-1,\,1]} (\omega + \gamma x) = \omega - |\gamma|$$

$$\omega - |\gamma| \geq \delta > 0, \qquad \alpha \geq 0, \quad \beta \geq 0$$

The numerical margin delta is one millionth in squared percentage-return units. Together with positive initial variance, these inequalities guarantee positive variance for every bounded sentiment input. The optimizer implements omega minus gamma and omega plus gamma each at least delta as two linear constraints.

This is sufficient for all allowed sentiment values, not necessary for positivity along a particular observed sequence. It is deliberately conservative and can materially restrict coefficients. It is a modeling revision, not merely an optimizer repair. In fact, many corrected fits touch this constraint. Alternatives include a positive link for the time-varying intercept or a model for log variance, but those would be different specifications and are not silently substituted here.

The corrected persistence cap is alpha plus beta at most 0.999. It keeps estimation away from the unit-persistence boundary. The number 0.999 is a numerical research choice rather than an economically estimated threshold. Results near any constraint need sensitivity analysis.

If the joint driver is stationary and ergodic, the innovation assumptions hold, and a finite mean exists, the unconditional mean identity becomes:

$$E[h_t] = \frac{\omega + \gamma E[x_t]}{1 - \alpha - \beta}$$

This identity does not establish stationarity of a nonstationary news process. Bounded sentiment and positive variance alone are insufficient. The term exogenous here describes an additional predictable regressor; the project does not establish strict econometric exogeneity or causal independence from market conditions.

9 Gaussian likelihood and quasi maximum likelihood

For Gaussian maximum likelihood, the model assumes the conditional return density is normal with mean mu and variance h_t. The contribution of observation t is:

$$f(R_t \mid \mathcal{F}_{t-1}; \theta) = \frac{1}{\sqrt{2\pi h_t}} \exp\left(-\frac{(R_t - \mu)^2}{2h_t}\right)$$

Taking minus logarithms converts products of densities into a sum. The negative log-likelihood minimized by the project is:

$$Q_T(\theta) = \frac{1}{2} \sum_{t=1}^{T} \left[\log(2\pi) + \log h_t + \frac{\varepsilon_t^2}{h_t}\right]$$

The logarithmic term penalizes unnecessarily large variances; the squared-residual-over-variance term penalizes variance that is too small for the observed shock. Removing either term would not estimate the same model. Mu enters both the current residual and every subsequent variance through the lagged squared residual, so it must be optimized jointly.

When conditional normality is unrealistic but the conditional mean and variance are correctly specified, the same objective is called Gaussian quasi maximum likelihood. Consistency and asymptotic inference require additional regularity, dependence, identification, and moment assumptions. The phrase robust standard errors does not make a misspecified conditional variance correct.

Financial returns often have heavy tails and asymmetric volatility responses. The current custom likelihood is Gaussian. A standardized Student-t innovation model or a leverage term would be useful future comparisons, but neither is estimated as part of the corrected custom experiment. Student-t should be standardized to unit variance if h_t is to retain its variance interpretation; an unstandardized scale parameter is not automatically variance. [S9]

The original package-based fitting_garch11.py uses the arch library, whereas the original comparison uses a custom recursion. Library defaults for backcasting, constraints, and fitting need not match the custom code. Therefore the library summary should not be assumed to supply the exact parameters used for held-out comparisons. The corrected forecast comparison uses one shared implementation for both models; the arch entry point remains explicitly descriptive.

10 Initialization optimization and numerical checks

The variance recursion needs an initial value. The corrected estimator uses the training-sample variance with denominator T, floored only at a tiny positive value for a degenerate training sample. It keeps that same initial value when filtering the complete observed path. This initialization is a fitted nuisance choice based solely on training observations.

$$h_1 = \max\left\{ \frac{1}{T_{\text{train}}} \sum_{t=1}^{T_{\text{train}}} (R_t - \bar{R}_{\text{train}})^2, \delta \right\}$$

The original forecast functions initialize from the entire return vector whenever called. In the test-stage call that includes future observations, so it is a technical leakage bug. The effect is geometrically attenuated: holding parameters and observed innovations fixed, two initial conditions differ at t by beta to the power t minus one times their original difference. For the long AAPL training path, the remaining influence at the split is effectively negligible. Fixing the information boundary remains correct even when that particular numerical effect is tiny.

The original optimizer is L-BFGS-B with box bounds and a single initial point. Separate alpha and beta bounds do not restrict their sum; the hard rejection penalty also creates flat invalid regions. An optimizer success flag alone is not evidence of a globally optimal or scientifically sensible fit.

The corrected implementation uses SLSQP because it accepts the linear positivity and persistence constraints. It optimizes average NLL for numerical scaling, then converts the reported objective back to total NLL. There are three deterministic baseline starts and ten extension starts: three persistence patterns crossed with three signed gamma choices, plus the fitted baseline embedded at gamma zero. [S10]

The baseline starts use alpha and beta pairs (0.05, 0.90), (0.10, 0.70), and (0.15, 0.30). Omega initially approximates training variance times one minus persistence; mu starts at the training mean. Extension gamma starts at zero or plus/minus half the initial omega. Mu and gamma are otherwise not arbitrarily restricted to the old boxes.

Only successful, finite, feasible solutions are retained, and the smallest NLL is selected. Feasibility is rechecked with numerical tolerance. Multiple starts improve practical reliability but do not prove a global optimum. For each fit, result files record parameter values, objective, convergence counts, boundary status, and variance-range diagnostics.

The recursion is computed with scipy.signal.lfilter, since conditional on a parameter vector and the observed residual sequence it is a first-order linear filter in h. A numerical test compares its output to a hand calculation. This is an implementation acceleration; it does not turn GARCH itself into a linear model in the observed returns.

11 Parameter uncertainty and the original robust covariance code

The original fitting_garch-x.py computes individual NLL contributions, a numerical Hessian of total NLL, numerical observation-level scores, a Hessian covariance, and a sandwich covariance. The intended scaling of its sandwich formula is reasonable for a score outer-product estimator. The primary concerns are regularity, constraint boundaries, finite-difference steps, conditioning, and the treatment of invalid covariance diagonals.

Let ell_t be a negative log-likelihood contribution and q_t its gradient. Define H as the Hessian of the total objective. At a regular interior solution under appropriate assumptions:

$$H = \sum_t \frac{\partial^2 \ell_t}{\partial\theta\partial\theta'}, \qquad J = \sum_t q_t q_t'$$

$$\widehat{\mathrm{Cov}}(\hat{\theta}) = H^{-1} J H^{-1}$$

Equivalently, with A equal to H divided by T and B equal to J divided by T, the covariance is A inverse times B times A inverse divided by T. If the scores are a martingale difference, contemporaneous outer products are appropriate under standard QMLE theory. If the required score structure fails and serial correlation remains, a long-run score covariance may be required instead.

A negative diagonal in an estimated covariance is evidence to diagnose. Replacing it with zero and then reporting a zero standard error can create infinite test statistics; using a pseudoinverse does not establish identification. Numerical perturbations near a positivity boundary can enter invalid regions, and the penalty values are not derivatives of a valid smooth likelihood.

Constrained estimates on a boundary generally do not have the same unrestricted normal limit used for ordinary Wald intervals. Because seven of the nine corrected main GARCH-X fits have a boundary flag, the replacement full-sample custom fit reports estimates and diagnostics without manufacturing unconstrained coefficient p-values. It does not claim to implement a validated constrained bootstrap or boundary-adjusted interval procedure.

For deeper implementation understanding, define g_t as the vector derivative of h_t, holding the training initialization fixed. With parameter order mu, omega, alpha, beta, gamma, the recursion gives these components:

$$\frac{\partial h_t}{\partial \mu} = -2\alpha\varepsilon_{t-1} + \beta\frac{\partial h_{t-1}}{\partial \mu}$$

$$\frac{\partial h_t}{\partial \omega} = 1 + \beta\frac{\partial h_{t-1}}{\partial \omega}, \qquad \frac{\partial h_t}{\partial \alpha} = \varepsilon_{t-1}^2 + \beta\frac{\partial h_{t-1}}{\partial \alpha}$$

$$\frac{\partial h_t}{\partial \beta} = h_{t-1} + \beta\frac{\partial h_{t-1}}{\partial \beta}, \qquad \frac{\partial h_t}{\partial \gamma} = x_t + \beta\frac{\partial h_{t-1}}{\partial \gamma}$$

$$q_t = -\frac{\varepsilon_t}{h_t} e_\mu + \frac{1}{2}\left(\frac{1}{h_t} - \frac{\varepsilon_t^2}{h_t^2}\right) g_t$$

Here e_mu selects the mean component. These derivatives explain what a numerical gradient approximates. A log-variance model, different initialization rule, or different mean model changes the derivatives. The optimizer uses numerical derivatives.

12 Chronological forecasting without future returns

The original study uses the earliest 80 percent for training and the last 20 percent for evaluation. It estimates parameters once and then updates the conditional variance state as new returns arrive. This is a fixed-parameter sequence of one-step forecasts, not a sequence of parameter refits and not a single forecast of all future days from one origin.

In the corrected main experiment, most stocks have 1,204 training returns and 301 evaluation returns, with evaluation from 25 March 2025 through 4 June 2026. META has a later usable start and therefore 988 training returns and 248 evaluation returns, beginning 10 June 2025. Exact dates are recorded in the metrics.

For the first evaluation return, use the last training return, its filtered variance, and the sentiment already available at its issuance. After that return is observed, update the state for the next forecast. Test returns may influence subsequent test forecasts; they must not influence an earlier forecast or the fixed training estimates.

```
Read sorted returns and target aligned sentiment
Choose a chronological split
Estimate parameters using the training segment only
Set h initial from training returns only
For each session t after the first
    h[t] = omega + alpha * (R[t-1] - mu)^2 + beta * h[t-1]
    For GARCH X add gamma * x[t]
    Record h[t] before using the realized R[t]
Evaluate the held out h[t] against a common proxy
```

Because h_t uses only the preceding return, filtering the whole observed vector at fixed parameters is equivalent to sequential computation, provided initialization is training-only and x_t is correctly aligned. A test changes future returns and verifies that forecasts up to the corresponding origin remain identical.

An expanding or rolling refit could adapt parameters to changing regimes, but it is a different forecasting procedure. It adds computation and parameter-estimation variability. Model comparison must specify not just the equation but also the estimation window, refitting frequency, initial state, and feature clock.

For ordinary GARCH, if parameters are fixed and the baseline assumptions hold, multi-step expected variance reverts geometrically toward the long-run mean. With GARCH-X, future sentiment expectations or explicit scenarios are needed; it is not legitimate to insert subsequently observed news into a forecast purportedly issued many days earlier.

$$E_t[h_{t+k}] = \bar{h} + (\alpha + \beta)^{k-1}(h_{t+1} - \bar{h}) \quad \text{for ordinary GARCH}$$

The project evaluates only sequential one-step variance forecasts. It does not evaluate multi-day cumulative-return risk, portfolio variance, implied volatility, or option prices.

13 Evaluation targets loss functions and benchmarks

Conditional variance is latent. The original comparison uses the squared residual around the baseline fitted mean. The corrected comparison uses one common training-sample mean for every model, making target construction independent of which model is being scored.

$$y_t = (R_t - \bar{R}_{\text{train}})^2$$

This is a noisy proxy, not observed true variance. If the true conditional mean is m_t, then its conditional expectation is h_t plus the squared difference between m_t and the fixed training mean. Consequently, the proxy is conditionally unbiased for variance only when the centering is correct. Using one common center improves comparability but does not establish that assumption.

The variance mean squared error and QLIKE losses are:

$$L_{\mathrm{MSE}}(y, h) = (y - h)^2$$

$$L_{\mathrm{QLIKE}}(y, h) = \frac{y}{h} + \log h, \qquad h > 0$$

MSE is expressed in fourth powers of percentage-return units because its arguments are variances. It can be dominated by a few unusually large squared returns. QLIKE penalizes underprediction relative to the predicted variance and remains defined when y is zero. The expression used here omits terms depending only on y from another common QLIKE convention; such terms do not change paired model rankings. [S11]

To see the conditional optimum, write v equal to the conditional expectation of y. Expected QLIKE is v divided by h plus log h, whose derivative is (h minus v) divided by h squared. Its minimum over positive h is v. Expected MSE is minimized by the same v. Thus these losses target conditional variance when y is a suitable unbiased proxy, rather than rescuing an incorrectly centered target.

$$\frac{\partial E[L_{\mathrm{QLIKE}} \mid \mathcal{F}]}{\partial h} = \frac{h - E[y \mid \mathcal{F}]}{h^2}$$

The proxy-noise argument can be seen directly. For any two forecasts measurable at the origin, the QLIKE loss difference is linear in y, so replacing a conditionally unbiased proxy by latent variance preserves its conditional expected difference. For MSE, the y squared terms cancel in a paired difference, leaving a term linear in y. This derivation concerns expected ranking under the stated assumptions, not finite-sample certainty. The relevant econometric treatment is Patton's study of imperfect volatility proxies. [S11]

$$L_Q(y, h_A) - L_Q(y, h_B) = y\left(\frac{1}{h_A} - \frac{1}{h_B}\right) + \log\left(\frac{h_A}{h_B}\right)$$

The corrected experiment adds two benchmarks. The constant forecast is the training-sample variance, held fixed throughout evaluation. EWMA uses a prespecified decay of 0.94 and the same training-based center and initial state:

$$h_t^{\mathrm{EWMA}} = 0.94 h_{t-1}^{\mathrm{EWMA}} + 0.06(R_{t-1} - \bar{R}_{\mathrm{train}})^2$$

The decay 0.94 is a fixed conventional benchmark setting, not a tuned optimum or a newly estimated parameter. A model can outperform GARCH-X's nested baseline while still losing to a simpler benchmark. All benchmark losses use the identical dates and proxy within a ticker.

QLIKE values can be negative in some units because of log h. A negative loss is not automatically an error. Adding or scaling a units-dependent constant changes reported levels; compare models within a consistently scaled dataset. A percentage reduction in QLIKE has no simple invariant economic meaning.

14 Diebold Mariano comparisons and multiple testing

For each ticker and each loss, define d_t as baseline GARCH loss minus GARCH-X loss. A positive average means GARCH-X has the smaller average loss. The null is equality of expected losses for the forecasting procedures under comparison, not gamma equal to zero and not absence of a causal news effect. [S12]

$$d_t = L(y_t, \hat{h}_t^{G}) - L(y_t, \hat{h}_t^{GX}), \qquad H_0 : E[d_t] = 0$$

$$DM = \frac{\bar{d}}{\sqrt{\widehat{\Omega}/T}}$$

The long-run variance Omega includes the contemporaneous variance of loss differences and their serial autocovariances. One-step forecast errors can still yield serially correlated loss differences through persistent states, model misspecification, or other dependence. Therefore setting the HAC bandwidth to horizon minus one gives no autocorrelation protection at horizon one.

The corrected implementation uses consistently demeaned sample autocovariances with denominator T, Bartlett weights, and a transparent bandwidth rule. [S13]

$$\hat{\gamma}_k = \frac{1}{T} \sum_{t=k+1}^{T} (d_t - \bar{d})(d_{t-k} - \bar{d})$$

$$\widehat{\Omega} = \hat{\gamma}_0 + 2 \sum_{k=1}^{L} \left(1 - \frac{k}{L+1}\right) \hat{\gamma}_k$$

$$L = \lfloor 4(T/100)^{2/9} \rfloor$$

This produces five lags for the 301-observation main evaluations and four for META's 248 observations. The rule is a bandwidth heuristic, not a proof that remaining dependence is absent. The function supports alternate bandwidths. Constant or numerically degenerate loss differences return an undefined statistic and p-value instead of a fabricated significant result.

Two-sided exploratory p-values use the standard normal survival function. Finite evaluation samples, heavy-tailed losses, estimated parameters, model nesting, and constraints can affect calibration. The corrected output labels these p-values exploratory. Adding HAC does not by itself solve nested-model forecast testing or boundary asymptotics. Giacomini and White provide a framework that explicitly treats forecasting methods and estimation schemes; a fully validated extension would require implementing and checking a suitable design rather than borrowing its name for the present statistic. [S14]

The corrected main family consists of eighteen comparisons, nine tickers times two losses. Holm's step-down adjustment orders valid p-values from smallest to largest, multiplies by the number of remaining hypotheses, takes cumulative maxima, and caps at one. Undefined comparisons are conservatively treated as p equal to one when setting family size. [S15]

$$p_{(i)}^{\mathrm{Holm}} = \min\left\{1, \max_{j \le i} (m - j + 1) p_{(j)}\right\}$$

Holm can control familywise error under arbitrary cross-hypothesis dependence when the underlying p-values are valid. It cannot correct a misspecified null distribution. Sensitivity runs have their own reported adjustment families and are exploratory; they are not eighteen newly independent confirmatory tests. Using multiple splits and feature choices after viewing outcomes would require a broader research design.

15 Empirical findings

15 1 Reproduction of the original experiment

The original twelve source scripts were archived before corrections. audit_original.py extracts only function definitions from the archived comparison script, avoiding its import-time merging and overwriting behavior. It reuses the original cached merged inputs and optimizer settings. All ten original forecast fits converged in this environment and produced valid observed variance paths.

Ticker	GARCH	GARCH X	G minus GX	Original p
AAPL	2.07705	2.09475	-0.01770	0.01715
MSFT	2.08330	2.10072	-0.01742	0.05779
NVDA	2.82483	2.81993	0.00489	9.66e-05
AMZN	2.49572	2.50278	-0.00706	0.71039
GOOGL	2.38449	2.38170	0.00279	0.86626
META	2.83899	2.84545	-0.00646	0.64178
BRK-B	1.14496	1.14643	-0.00147	0.46637
LLY	3.17074	3.23067	-0.05993	0.23192
AVGO	3.23117	3.21321	0.01796	0.27295
TSLA	3.32484	3.32570	-0.00086	0.34021

Table 3. Original QLIKE comparison. Positive difference favors GARCH-X. Original p-values use the original zero-lag normal-reference calculation and have no multiplicity adjustment. They are reproduced for audit, not endorsed as corrected evidence.

Original GARCH-X has lower average QLIKE for NVDA, GOOGL, and AVGO. This is three of ten stocks. Forecast superiority cannot be inferred from coefficient significance.

Verified QLIKE count: 3 of 10 stocks favor GARCH-X on average. The stocks are NVDA, GOOGL, AVGO. This count describes average loss, not a count of statistically validated discoveries.

15 2 Corrected main experiment

The corrected experiment changes several factors jointly: actual issuance timing, duplicate treatment, training initialization, parameter restrictions, sample eligibility, return inclusion, mean centering of the target, and test covariance estimation. Consequently, changes from Table 3 to Table 4 cannot be attributed to one isolated repair.

Ticker	GARCH	GARCH X	EWMA	G minus GX	HAC p	Holm p
AAPL	2.07508	2.07612	2.08656	-0.00104	0.40788	1.00000
MSFT	2.07421	2.07583	2.07724	-0.00162	0.16865	1.00000
NVDA	2.82966	2.82520	2.77308	0.00446	0.00136	0.02445
AMZN	2.49052	2.48612	2.59123	0.00441	0.34229	1.00000
GOOGL	2.39066	2.38748	2.45634	0.00318	0.23985	1.00000
META	2.71082	2.71157	2.92756	-0.00075	0.94441	1.00000
LLY	3.13449	3.17812	3.04055	-0.04362	0.26131	1.00000

Ticker	GARCH	GARCH X	EWMA	G minus GX	HAC p	Holm p
AVGO	3.23763	3.23642	3.25811	0.00121	0.76064	1.00000
TSLA	3.32906	3.33075	3.29517	-0.00169	0.19705	1.00000

Table 4. Corrected main QLIKE results. Lower loss is better; positive G minus GX favors the extension. Holm values refer to the declared eighteen-comparison main family and remain exploratory.

GARCH-X has smaller main average QLIKE for NVDA, AMZN, GOOGL, and AVGO, four of nine eligible stocks. Small differences rounded in a printed table should not be interpreted as exact equality.

Verified QLIKE count: 4 of 9 stocks favor GARCH-X on average. The stocks are NVDA, AMZN, GOOGL, AVGO. This count describes average loss, not a count of statistically validated discoveries.

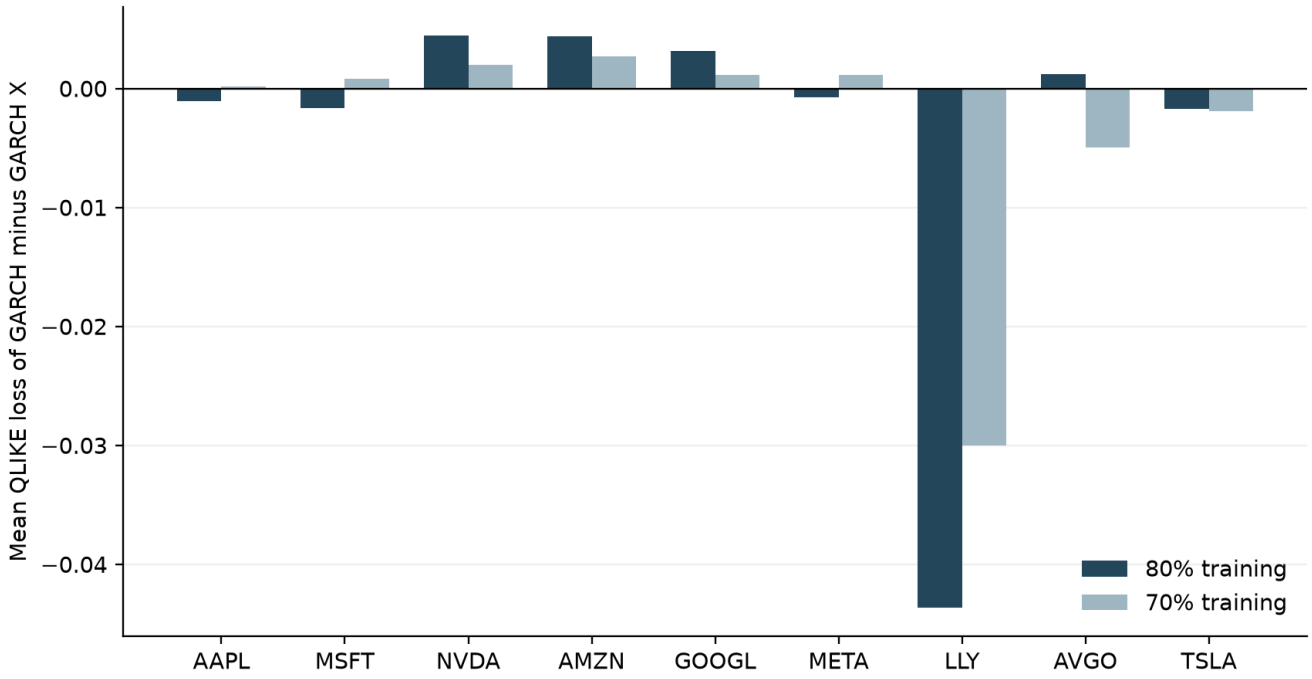


Figure 1. Signed average QLIKE differences for the corrected main and 70 percent training splits. Values above zero favor GARCH-X. The split comparison changes training and test periods and is not a pair of independent replications.

NVDA's main QLIKE is about 2.82966 for GARCH and 2.82520 for GARCH-X. Its exploratory HAC p-value is about 0.00136 and its main-family Holm value is about 0.02445. However, EWMA's QLIKE is about 2.77308, which is smaller than both. The public conclusion should therefore be specific: the sentiment extension improves on this GARCH baseline for NVDA under the investigated loss and splits, while it does not provide the best benchmark forecast in the main comparison.

LLY illustrates the difference between training fit and forecasting. Its GARCH-X likelihood improves in training and its fitted beta is near zero, yet its main QLIKE is worse than baseline. A flexible variance equation can fit historical observations better and generalize worse. A dramatic parameter change is a reason to inspect model stability rather than a reason to celebrate an extra feature.

Ticker	GARCH	GARCH X	EWMA	Constant
AAPL	184.290	184.273	184.764	191.374
MSFT	90.001	90.018	90.915	90.020
NVDA	344.817	344.056	343.483	371.777
AMZN	146.965	146.425	148.323	147.531

Ticker	GARCH	GARCH X	EWMA	Constant
GOOGL	94.401	94.298	96.358	94.199
META	262.466	260.548	242.242	246.755
LLY	467.375	468.172	471.560	469.455
AVGO	646.750	645.176	646.915	653.144
TSLA	886.439	886.659	893.625	935.071

Table 5. Corrected mean squared errors of daily variance proxies and forecasts. Units are percentage-return units to the fourth power. Rankings can differ from QLIKE because losses weight forecast errors differently.

15 3 Sensitivity analyses

Ticker	Main diff	70 percent diff	Coverage diff	Coverage test N
AAPL	-0.00104	0.00017	-9.76e-06	172
MSFT	-0.00162	0.00079	-0.00162	301
NVDA	0.00446	0.00200	0.00446	301
AMZN	0.00441	0.00268	0.00441	301
GOOGL	0.00318	0.00114	0.00318	301
META	-0.00075	0.00117	-0.01420	168
LLY	-0.04362	-0.02999	-0.04362	301
AVGO	0.00121	-0.00496	0.00121	301
TSLA	-0.00169	-0.00191	-0.12210	113

Table 6. QLIKE differences across chronological and coverage sensitivities. The 70 percent training analysis has a longer, overlapping test period. The coverage sensitivity retains only the final contiguous segment after the last observed news gap over 30 calendar days, where such a gap exists.

The alternative training split does not establish broad sentiment superiority. The coverage screen changes both sample size and calendar composition, particularly for AAPL, META, and TSLA. Its results demonstrate why a single global claim based on the cached collection would be too strong. Even the screened sample does not prove complete data capture or historical text availability.

In the coverage-screened TSLA experiment, the signed QLIKE difference is approximately minus 0.12210, favoring baseline GARCH by considerably more than in the main experiment. Its normal-reference statistic is very large in magnitude, but that remains conditional on the short 113-observation evaluation, parameter constraints, loss dependence, and the selected coverage screen. It should not be elevated into a universal anti-sentiment claim. The final screened training segments begin in January 2023 for AAPL, February 2023 for META, and March 2024 for TSLA.

16 Parameters diagnostics and interpretation

Ticker	mu	omega	alpha	beta	gamma	a plus b	Boundary
AAPL	0.09760	0.03601	0.03857	0.94981	0.03601	0.98838	True
MSFT	0.08189	0.01842	0.02551	0.96729	0.01842	0.99280	True

Ticker	mu	omega	alpha	beta	gamma	a plus b	Boundary
NVDA	0.27154	0.70827	0.05814	0.87905	-0.29652	0.93720	False
AMZN	0.07147	0.44507	0.15629	0.76379	-0.44507	0.92008	True
GOOGL	0.08669	0.04572	0.01457	0.97312	-0.04572	0.98770	True
META	0.07484	1.45886	0.22385	0.66437	-1.45886	0.88821	True
LLY	0.14351	3.79756	0.02056	0.00000	-1.43742	0.02056	True
AVGO	0.16102	0.08032	0.07072	0.92084	0.08032	0.99155	True
TSLA	0.13592	0.45433	0.04540	0.92713	-0.19069	0.97253	False

Table 7. Corrected main GARCH-X training estimates. Persistence is alpha plus beta. Boundary flags include near-zero alpha or beta, the persistence cap, or the positivity margin. Coefficient values are not standardized effect sizes.

Seven corrected main GARCH-X estimates have a boundary flag: AAPL, MSFT, AMZN, GOOGL, META, LLY, and AVGO. For all except LLY in this list, the signed-intercept positivity margin is active; LLY has beta approximately zero. This shows the substantive effect of the globally safe signed-intercept restriction. Parameter uncertainty must respect these constraints.

Standardized residuals divide fitted innovations by the square root of their conditional variances. A useful model should reduce predictable variance structure in squared standardized residuals. The implementation records training-sample Ljung-Box checks at ten lags for standardized residuals and their squares, and a ten-lag ARCH LM check. These are approximate diagnostic p-values, not proof that the full innovation distribution is correct.

$$\hat{z}_t = \frac{R_t - \hat{\mu}}{\sqrt{\hat{h}_t}}$$

Ticker	LB z p	LB z squared p	ARCH LM p	Minimum h
AAPL	0.98765	0.85955	0.82127	1.27252
MSFT	0.40985	0.95785	0.93580	1.18118
NVDA	0.42203	0.99895	0.99892	6.31633
AMZN	0.82564	0.73195	0.73305	1.81309
GOOGL	0.37282	0.93646	0.93152	2.32007
META	0.85630	0.99991	0.99989	3.73277
LLY	0.84660	0.90932	0.85486	2.45680
AVGO	0.63171	0.52976	0.52207	1.50469
TSLA	0.03816	0.16345	0.18473	7.13215

Table 8. Corrected main GARCH-X training diagnostics. Large p-values mean these tests do not detect the particular dependence checked; they do not establish Gaussian tails, good risk calibration, or complete specification. Parameter estimation and multiple diagnostics complicate nominal interpretations.

TSLA has a standardized-residual Ljung-Box p-value near 0.038 at ten lags, suggesting that the constant mean deserves investigation. Squared-residual tests are less concerning for the fitted training sample, but that does not eliminate potential tail misspecification. A full diagnostic study should inspect residual plots, autocorrelations, quantile plots, tail exceedances, and regime stability with an appropriate validation plan.

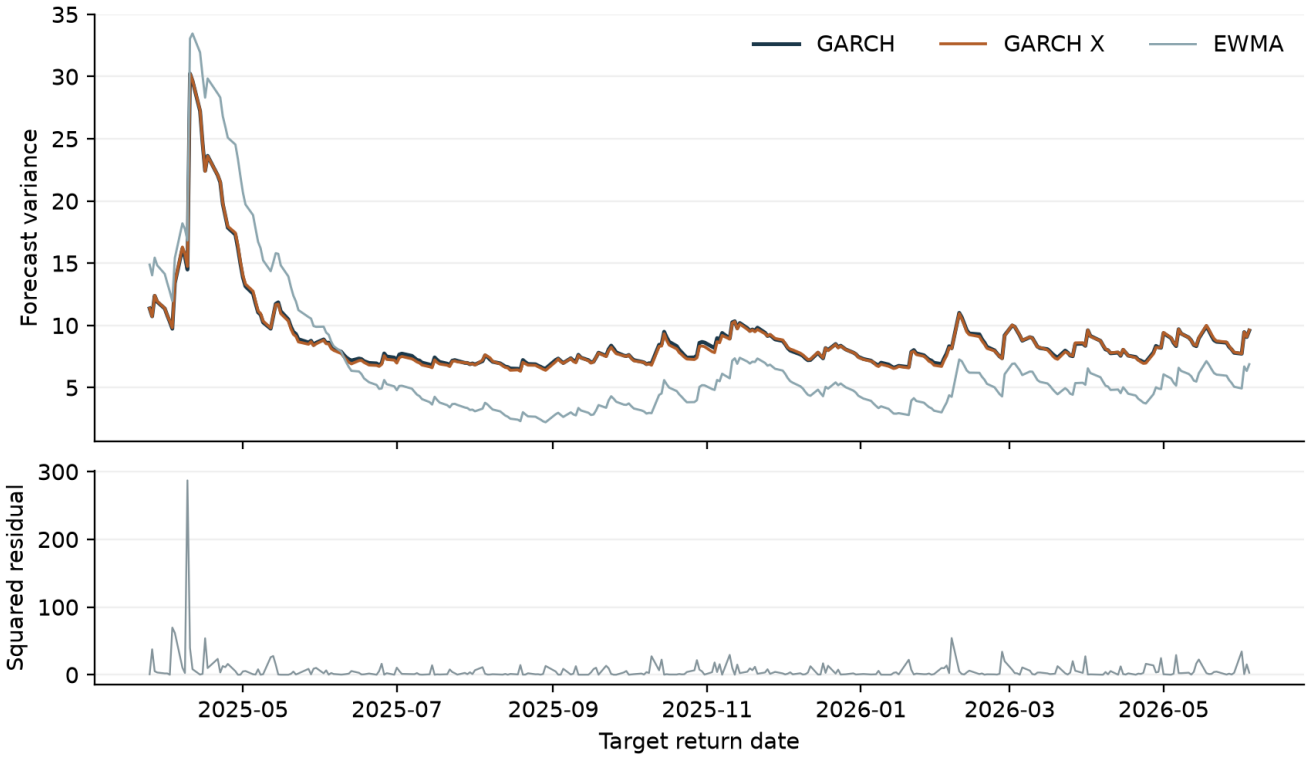


Figure 2. NVDA main test variance forecasts above and its noisy squared-residual proxy below. Both panels use squared percentage-return units, with separate vertical scales. Smooth conditional variance curves need not match every daily spike. The plot is a variance comparison, not a prediction of return direction.

Information criteria are also written to corrected_fits.json. AIC is twice parameter count plus twice total NLL; BIC replaces the parameter penalty with log of sample size times parameter count. They compare penalized in-sample fit on the same observations and objective, not out-of-sample performance. Near constraints, their usual asymptotic interpretation also deserves caution.

17 Implementation map

17 1 Data and feature modules

project_config.py contains ticker names, paths, the variance margin, persistence cap, issuance delay, and the BRK-B exclusion reason. Importing it performs no data work. corrected_data.py reads the existing price cache, computes log differences, constructs exchange issuance times, reads and deduplicates sentiment, assigns news to issuance windows, records counts, and writes corrected frames. merge_data.py delegates to this implementation when executed.

The feature frame's Date is the target return session. log_return is in decimal units and is multiplied by 100 in forecasting. sentiment_lag1 is already x_t. forecast_origin identifies the preceding session's issuance time, and news_window_start identifies the open left endpoint of the preceding issuance window. coverage_envelope is an eligibility screen on the endpoints. realized_vol is retained only for descriptive inspection.

yfinancedownload.py is now a safe cache-inspection entry point. It does not silently call Yahoo on import. A future price refresh must deliberately specify adjustment settings and a new snapshot destination. get_news.py is the explicit optional news refresh. sentiusingpytorch.py provides lazy inference, while run_sentiment.py is an optional rescoring path that writes a new directory and records the checkpoint metadata it can retrieve.

The old lag_sentiment.py has been retired (it remains in the original-source archive); merge_data.py uses the exchange-calendar construction. The old version shifted only the dates that happened to contain news, so one row could mean several calendar days and was inconsistent with the modeling merge. Old daily_sentiment CSVs and the old exploratory notebook remain historical artifacts; the corrected modeling path does not consume them.

17 2 Estimation and forecasting modules

volatility_models.py implements array validation, the variance recursion, NLL contributions, constrained multi-start fitting, QLIKE, the exploratory HAC comparison, and Holm adjustment. corrected_forecasts.py handles chronological splitting, optional coverage screening, sequential fixed-parameter forecasts, common targets, benchmark losses, diagnostics, and result export. forecast_comparision.py preserves the original filename as a thin entry point to the corrected experiment.

fitting_garch11.py fits a descriptive full-sample baseline using the arch package. fitting_garch-x.py fits the descriptive constrained custom extension and reports boundary information. Neither should be substituted for training-only estimation when reporting held-out forecasts. insight.py now plots available corrected columns with the proper pyplot import; the original placeholder column names and literal f-string path would fail.

audit_original.py reproduces the archived comparison without running import-time side effects. test_project.py contains behavioral checks that address mathematical correctness and timing rather than merely echoing implementation statements. The handout's tables and figures are generated from machine-readable outputs, limiting transcription errors.

17 3 Data lineage and output files

Root-level raw and old merged CSVs are unchanged. input_data_sha256.json records their hashes at audit start. original_source_before_corrections.zip preserves the twelve original Python files. original_metrics.csv, original_fits.json, and data_audit.csv document the reproduction and data audit. corrected_data contains new feature frames; each experiment produces metrics, fitted-parameter metadata, and ticker-level forecast CSVs.

The result prefixes are corrected for the main 80 percent training experiment, sensitivity70 for the 70 percent training experiment, and coverage30 for the final-segment coverage screen. Each forecast file contains the common proxy, four forecasts, origin timestamp, and sentiment input. Each metrics file contains dates, sample sizes, mean losses, signed differences, HAC statistics, exploratory p-values, bandwidth, and its within-run Holm adjustment.

environment.json records the Python and relevant package versions used for the rerun. requirements.txt pins the analysis dependencies from that environment. requirements-optional.txt covers downloading, inference, and document-generation tools. Torch is deliberately optional because cached-score analysis does not need it, and a suitable Torch build depends on the target machine.

18 Corrections made and their scope

The corrections remove execution on import from the modeling and data entry points, centralize configuration, repair the broken sentiment-module import, make GPU inspection safe when CUDA is absent, and replace the exploratory plot's invalid path and placeholder columns. They preserve all original source in an archive.

For modeling, they use explicit exchange-session timestamps and a forecast origin, remove duplicate cached headlines by a stated approximation, retain news counts, screen the observed news envelope, and exclude BRK-B's incomplete end coverage. Returns no longer depend on an unused rolling-volatility column. The main experiment still uses the supplied cache; missing historical records are not invented.

For estimation and evaluation, they impose sufficient signed-intercept positivity, a persistence cap, deterministic multiple starts, convergence and feasibility checks, and a training-only initial variance carried into evaluation. They use a common training center for the proxy, add simple benchmarks, estimate autocorrelation-aware loss covariance, handle degenerate comparisons, adjust the declared comparison family, and export inspectable forecasts and diagnostics.

The original custom covariance output is replaced by transparent estimates and diagnostics because unrestricted Wald summaries at constrained boundaries would be misleading. This is not a claim that statistical uncertainty is solved; it is an explicit limit on what the present output warrants.

The public-facing package includes no credentials or raw headlines. The local .env remains private and unchanged. The added ignore rules and placeholder example support future source sharing. Data redistribution permissions are separate from permission to analyze a locally cached dataset; sharing the report does not require bundling the underlying licensed news text.

Not every desirable extension was implemented. There is no prospective archive of first-seen text, validated constrained-parameter inference, full nested-model forecast bootstrap, rolling-refit study, Student-t custom likelihood, market-wide sentiment control, intraday variance estimator, or economic trading evaluation. These are identified as future work rather than reported as completed repairs.

19 Reproduction and verification

Use Python 3.13 with the pinned analysis dependencies. Run these commands from the project directory. They use existing local caches and do not require API credentials or a GPU.

```
python -m pip install -r requirements.txt
python -m unittest test_project -v
python merge_data.py
python forecast_comparision.py
python corrected_forecasts.py --train-fraction 0.7 --label sensitivity70
python corrected_forecasts.py --screen-long-gaps 30 --label coverage30
python audit_original.py
```

Run experiment commands sequentially when reproducing the whole collection so the same output destination is not written by multiple processes. Different labels create separate results. The archived-original reproduction deliberately reads the original merged CSVs; it should not be pointed at corrected frames and called an original reproduction.

The eight behavioral tests cover a hand-calculated signed-sentiment recursion; no influence of future returns on earlier forecasts when initialized from training; early closes and daylight saving; weekend and exact-cutoff assignment; positivity and persistence restrictions under extreme future sentiment; exact nesting at gamma zero; QLIKE input validation and degenerate comparisons; and safe imports that do not modify root CSVs.

The analysis reruns completed for all nine eligible stocks under all three corrected designs. The original reproduction completed for ten stocks. The original CSV hashes are checked again before delivery. Optional network collection and FinBERT inference are not executed as part of this cached-data validation; those paths need credentials, a compatible Torch environment, and a new collection provenance record.

The PDF and Markdown sources are generated from the same handout. Public-facing tables are aggregates, and no article text, API key, or personal environment file is embedded. The package contains mathematical discussion, aggregate outputs, implementation source, tests, and setup instructions. It is prepared for public viewing; it has not been published to an external site or repository.

20 Interview answers and self assessment

What did you build

I built a study of one-trading-day conditional variance forecasting for selected US equities. I compared GARCH(1,1) with a signed-news-sentiment GARCH-X extension, using daily price returns and cached FinBERT headline scores. I estimated parameters chronologically and compared sequential held-out forecasts under QLIKE and variance MSE. Results were mixed, and the audit exposed material news-coverage and information-timing limitations.

Why forecast variance rather than returns

Variance models the conditional scale of uncertainty rather than the expected sign of a price change. Daily return direction may be difficult to predict while squared shocks exhibit persistence. GARCH explicitly models that persistence. Taking the square root produces volatility, but a variance forecast is not itself a directional trading signal.

What exactly is known at issuance

At the post-close origin, completed daily returns and news timestamped no later than that origin are eligible. The corrected feature uses the immediately preceding issuance window. Weekend news cannot enter a forecast issued on Friday. The historical cache lacks first-seen text versions, so perfect historical availability is not established.

Why use a lagged feature

The lag aligns information with the next return and reduces direct use of news generated during the target outcome. Calendar alignment matters more than the word lag: a one-row shift of arbitrary news dates or a full UTC day is not sufficient to establish availability at a market-close origin.

Is alpha plus beta less than one a positivity condition

No. Ordinary GARCH positivity follows from positive omega, nonnegative alpha and beta, and positive initial variance. Alpha plus beta less than one concerns a finite unconditional second moment under the usual assumptions. Signed sentiment requires an additional restriction or a positive model construction. This corrected implementation uses omega greater than absolute gamma by a positive margin.

Why can gamma be negative

The feature is signed tone. Negative gamma means positive tone lowers conditional variance and negative tone raises it, holding the state fixed. It does not make the total variance negative when the positivity restrictions hold. Its coefficient is an association within a specified recursion, not a causal effect of changing public sentiment.

Why not square sentiment

Squaring removes direction and makes large positive and negative scores contribute similarly. That tests a magnitude hypothesis. The original implementation tests a signed-tone hypothesis. Comparing both would require a declared model family and validation plan rather than rewriting the equation after seeing which one wins.

Why multiply returns by one hundred

It improves numerical scale for optimization. It is not z-score normalization. Variance parameters and forecasts change by a factor of ten thousand; alpha and beta do not. Interpretation and evaluation must use consistent units.

Why use Gaussian likelihood for heavy tailed returns

It is a tractable baseline and can be interpreted as quasi likelihood when conditional moments are correctly specified and regularity conditions hold. That does not guarantee robust tails or valid Gaussian prediction intervals. A Student-t likelihood and leverage model would be useful explicit extensions.

Does a better training likelihood establish added value

No. The extension nests the baseline and has an extra parameter, so better training fit can reflect flexibility. Held-out losses and stability provide a stronger check. LLY is a concrete example in this experiment where improved training fit does not yield better main QLIKE.

What does the Diebold Mariano test test

It tests whether the expected difference between two forecast-loss sequences is zero under its assumptions. It does not directly test gamma, causality, profitability, or whether one model is universally correct. The statistic's sign depends on the chosen subtraction order.

Why include autocovariances for one day forecasts

The loss differences can be serially dependent even at a one-day horizon. Forecast horizon alone does not identify the covariance bandwidth. The corrected test uses a Bartlett HAC estimator with an explicit bandwidth rule, but its normal-reference p-values remain exploratory.

What did the results show

The results do not support general sentiment superiority. The extension improves some ticker-loss combinations, and NVDA's QLIKE comparison is the clearest exploratory result. Yet an EWMA benchmark performs better than both GARCH models for NVDA in the main sample. Coverage defects and sample sensitivity limit broader conclusions.

What would you improve first

I would rebuild a verifiable news collection with article IDs, token pagination, first-seen and update histories, recorded model revisions, and coverage checks. Then I would preregister feature variants and forecasting procedures, use a fresh evaluation period, and implement inference suited to the chosen nested constrained forecasting design. Data provenance comes before adding model complexity.

Appendix A A complete numerical example

Suppose yesterday's daily percentage log return is 1.2, the mean is 0.2, yesterday's variance is 2, and yesterday's window sentiment is minus 0.8. Choose omega 0.6, alpha 0.1, beta 0.7, and gamma 0.5. These parameters satisfy the signed-intercept positivity margin and have persistence 0.8.

The innovation is 1.2 minus 0.2, or 1.0. Its square is 1.0. The shock contribution is 0.1, the lagged-state contribution is 1.4, the sentiment contribution is minus 0.4, and the intercept is 0.6. The next conditional variance is therefore 1.7 squared percentage-return units.

$$h_t = 0.6 + 0.1(1.0)^2 + 0.7(2.0) + 0.5(-0.8) = 1.7$$

Daily volatility is approximately 1.304 percentage points, and its conventional annualized display is about 20.70 percent. Without sentiment, holding these other values fixed, the variance would be 2.1. This is a local calculation, not a comparison of two separately reestimated models.

If sentiment were squared instead, its contribution would be plus 0.32 rather than minus 0.4. That would produce variance 2.42 with the same other terms, illustrating why the mathematical feature definition matters.

Suppose the realized common squared-residual proxy is 4. A variance forecast of 1.7 has MSE (4 minus 1.7) squared, or 5.29. Its QLIKE is 4 divided by 1.7 plus log 1.7, approximately 2.884. These are losses against a noisy observation; a forecast need not equal each day's realized squared residual to be a good conditional variance forecast.

For a safe worst-case argument, omega minus absolute gamma is 0.1. Therefore even the most adverse signed-sentiment contribution leaves the intercept component positive, and nonnegative shock and state contributions preserve positivity. The persistence condition plays a different role in long-run behavior.

Appendix B Notation and units

Symbol	Meaning	Units or domain
t	Trading-session index	Ordered exchange sessions
P_t	Cached closing price	Price units
r_t	Decimal log return	Dimensionless decimal
R_t	One hundred times log return	Percentage-return units
mu	Constant conditional mean	Percentage-return units
epsilon_t	Return minus model mean	Percentage-return units
h_t	Conditional variance	Squared percentage-return units
z_t	Standardized innovation	Conditional mean zero and variance one
omega	Variance intercept	Squared percentage-return units
alpha	Squared-shock coefficient	Nonnegative dimensionless
beta	Variance-state coefficient	Nonnegative dimensionless
gamma	Signed-sentiment coefficient	Squared percentage-return units
s_i	Article probability difference	Between minus one and one
x_t	Target-aligned mean sentiment	Between minus one and one
n_t	Articles in the issuance window	Nonnegative count
C_t	Close plus five-minute issuance	Timezone-aware timestamp
y_t	Common squared-residual proxy	Squared percentage-return units
d_t	GARCH loss minus GARCH-X loss	Loss-specific units
delta	Strict positivity margin	Squared percentage-return units

Appendix C Questions for further work

A strong next study would ask whether tone, attention, disagreement, and uncertainty have different predictive roles; whether market-wide news controls absorb the apparent stock-specific effect; whether the same result survives different losses and fresh regimes; and whether a more direct intraday variance proxy changes rankings. These are separable hypotheses and should be specified before selecting a winner.

It would also examine symbol history and vendor tagging. META's historical news coverage and BRK-B's symbol formatting may require provider-specific investigation; the present cache does not prove the cause of either coverage problem. A researcher should verify mappings rather than infer that a missing ticker feed means no relevant company news existed.

For implementation development, analytic gradient checks, simulation recovery under known parameters, and a validated boundary-aware uncertainty procedure would deepen confidence. The current eight tests verify essential software behavior, not consistency of the estimator under every data-generating process or calibration of all inferential statements.

References

[S1] Bollerslev T 1986. Generalized autoregressive conditional heteroskedasticity. Journal of Econometrics 31, 307-327. Author publication record and DOI: https://public.econ.duke.edu/~boller/research ; https://doi.org/10.1016/0304-4076(86)90063-1

[S2] Araci D 2019. FinBERT Financial Sentiment Analysis with Pre trained Language Models. https://arxiv.org/abs/1908.10063

[S3] ProsusAI. FinBERT checkpoint configuration and model information. https://huggingface.co/ProsusAI/finbert/blob/main/config.json

[S4] Alpaca. Historical news endpoint documentation. https://docs.alpaca.markets/us/reference/news-3

[S5] Alpaca. Official Python SDK news client source. The audit also inspected the installed RESTClient pagination implementation in alpaca-py 0.43.5. https://github.com/alpacahq/alpaca-py/blob/master/alpaca/data/historical/news.py

[S6] NYSE. Holidays and trading hours. https://www.nyse.com/trade/hours-calendars

[S7] Exchange calendars. Official project documentation and source. https://github.com/gerrymanoim/exchange_calendars

[S8] yfinance. Official download implementation and adjustment arguments. https://github.com/ranaroussi/yfinance/blob/main/yfinance/multi.py

[S9] arch. Official univariate volatility modeling documentation, distributions, and robust covariance examples. https://arch.readthedocs.io/en/stable/univariate/univariate_volatility_modeling.html

[S10] SciPy. SLSQP optimizer documentation. https://docs.scipy.org/doc/scipy/reference/optimize.minimize-slsqp.html

[S11] Patton A J 2011. Volatility forecast comparison using imperfect volatility proxies. Journal of Econometrics 160, 246-256. Author-hosted manuscript: https://public.econ.duke.edu/~ap172/Patton_robust_JoE_forthcoming.pdf

[S12] Diebold F X and Mariano R S 1995. Comparing predictive accuracy. Journal of Business and Economic Statistics 13, 253-263. https://users.ssc.wisc.edu/~behansen/718/DieboldMariano1995.pdf

[S13] Statsmodels. Newey West HAC covariance documentation and bandwidth convention. https://www.statsmodels.org/stable/generated/statsmodels.stats.sandwich_covariance.cov_hac.html

[S14] Giacomini R and White H 2006. Tests of conditional predictive ability. Econometrica 74, 1545-1578. https://doi.org/10.1111/j.1468-0262.2006.00718.x

[S15] Statsmodels. Multiple-testing adjustment documentation including Holm. https://www.statsmodels.org/stable/generated/statsmodels.stats.multitest.multipletests.html

Web documentation was checked on 21 October 2026. The mathematical derivations and worked examples in this handout are explanatory developments for the displayed project specification. Aggregate empirical statements refer to the locally reproduced outputs rather than results reported by those references.